JOSÉ MANUEL ORTEGA CANDEL

DESARROLLO SEGURO
EN INGENIERÍA DEL SOFTWARE

APLIACIONES SEGURAS CON ANDROID, NODEJS, PYTHON Y C++



CONTENIDOS WEB

Marcombo 1945-2020 75 años Tecnología, Ciencia y Formación

alphaeditorial

Desarrollo seguro en ingeniería del software

Aplicaciones seguras con Android, NodeJS, Python y C++

José Manuel Ortega Candel

Acceda a www.marcombo.info
para descargar gratis
contenido adicional
complemento imprescindible de este libro

Código: SOFTWARE1

Desarrollo seguro en ingeniería del software

Aplicaciones seguras con Android, NodeJS, Python y C++

José Manuel Ortega Candel

Marcombo 1945-2020 75 años

alphaeditorial

ALPHA EDITORIAL
ALFAOMEGA COLOMBIANA S.A.
Calle 62 No.20-46 esquina, Bogotá
Teléfono (57-1) 746 0102 Fax: (57-1) 210 0122
cliente@alfaomegacolombiana.com
www.alfaomega.com.co
www.alpha-editorial.com

Primera edición: Madrid, 2020
Bogotá, 2020

© José Manuel Ortega Candel
© Alpha editorial
© Alfaomega Colombiana S.A.
© Marcombo S.L.,

Todos los derechos son reservados. Esta publicación no puede ser reproducida total ni parcialmente. No puede ser registrada por un sistema de recuperación de información, en ninguna forma ni por ningún medio, sea mecánico, fotoquímico, electrónico, magnético, electroóptico, fotocopia o cualquier otro, sin el permiso previo y por escrito de la editorial.

Correctores: Mónica Muñoz y Beatriz García
Directora de producción: M.a Rosa Castillo
Diseño de la cubierta: ArteMio
Maquetación: xpress.cat

ISBN: 978-958-778-638-5 (Colombia)
ISBN: 978-84-267-2800-5 (España)

Hecho en Colombia
Printed and made in Colombia

Este libro va dedicado a aquellos que me han seguido,
me siguen y me seguirán, no importa cuál sea el camino escogido,
en algún lugar nos encontraremos.

Índice

Capítulo 1. Introducción al desarrollo seguro **15**

1.1 Propiedades del software seguro **17**
1.2 Principios de diseño seguro de aplicaciones **18**
1.2.1 Minimizar el área de la superficie de ataque **19**
1.2.2 Seguridad por defecto **19**
1.2.3 Privilegios mínimos **19**
1.2.4 Validación de datos de entrada **20**
1.2.5 Defensa en profundidad **20**
1.2.6 Control seguro de errores **20**
1.2.7 Separación de funciones **21**
1.2.8 Evitar la seguridad por oscuridad **21**
1.3 Análisis de requisitos de seguridad **21**

Capítulo 2. Aspectos fundamentales de desarrollo seguro **23**

2.1 Controles proactivos **24**
2.2 OWASP (Open Web Application Security Project) **25**
2.3. OWASP Mobile Security Project **29**
2.4 Controles proactivos OWASP **31**
2.4.1 Verificación de la seguridad desde las primeras etapas de desarrollo **32**
2.4.2 Validación de las entradas del cliente **32**
2.4.3 Desbordamientos del búfer **35**
2.4.4 Gestión de sesiones **35**
2.4.5 Implementación de controles de acceso **36**
2.4.6 Implementación de controles de identidad y autenticación **36**
2.4.7 Autenticación por múltiples factores **37**
2.4.8 Manejo de errores y excepciones **38**
2.5 Ataques en aplicaciones web **39**
2.5.1 Vectores de ataque **39**
2.5.2 Cross-site scripting (XSS) **40**
2.5.3 Cross-site request forgery (CSRF) **42**
2.5.4 Seguridad en las redirecciones **44**
2.6 SQL Injection: parametrización de las consultas en bases de datos **45**
2.6.1 Introducción a SQL Injection **46**
2.6.2 Problemas que pueden causar este tipo de ataques **47**
2.6.3 Ejemplo de inyección de SQL **48**
2.6.4 Escapar caracteres especiales utilizados en las consultas SQL **50**
2.6.5 Delimitación de los valores de las consultas **51**
2.6.6 Uso de sentencias preparadas parametrizadas **52**
2.6.7 Uso de procedimientos almacenados **53**

2.7 Seguridad en AJAX **55**

Capítulo 3. Herramientas OWASP **63**

3.1 DefectDojo **63**
3.2 SonarQube **69**
3.2.1 El cuadro de mando de SonarQube **70**
3.2.2 Issues por nivel de criticidad **72**
3.2.3 Perfiles de calidad **73**
3.2.4 Reglas SonarQube **76**
3.2.5 Informes de seguridad en SonarQube **78**
3.2.6 SonarQube Plugins **79**
3.2.7 Vulnerabilidades más comunes y explotadas **83**
3.3 Find Security Bugs **83**
3.3.1 Inyección potencial de Android SQL **85**
3.3.2 Abrir un socket sin cifrar **85**
3.4 LGTM **86**
3.5 OSS Index **89**
3.6 Snyk **91**
3.7 Otras herramientas de análisis estático **92**
3.8 Checklist de seguridad **93**

Capítulo 4. Seguridad en aplicaciones Android **95**

4.1 Introducción al protocolo HTTPS **95**
4.1.1 Conceptos básicos sobre certificados **95**
4.1.2 Despliegues en producción **97**
4.1.3 Certificado de servidor autofirmado **100**
4.1.4 CA no encontrada dentro de la cadena de certificados **100**
4.1.5 Configuración de seguridad **101**
4.1.6 Actualización de proveedores criptográficos **102**
4.1.7 Android Certificate Pinning **102**
4.1.8 Cifrado extremo a extremo **104**
4.1.9 Firmando una aplicación Android **104**
4.2 Principios fundamentales de desarrollo en Android **106**
4.2.1 Componentes en Android **107**
4.2.2 Android Lint **109**

4.3 Ingeniería inversa en Android **113**
4.3.1 ADB (Android Debug Bridge) **114**
4.3.2 Dex2jar **114**
4.3.3 JD-GUI **116**
4.3.4 jadx - Dex to Java decompiler **116**
4.3.5 Apktool **118**
4.3.6 Código smali y Mobylizer **121**
4.3.7 Androwarn **123**
4.3.8 Mobile Security Framework (MobSF) **125**
4.3.9 ClassyShark **127**
4.3.10 Drozer **128**
4.3.11 QARK **134**

4.3.12 SanDroid **137**
4.3.13 Yaazhini **138**
4.4 Buenas prácticas de desarrollo seguro en Android **139**
4.4.1 Seguridad en AndroidManifest.xml **140**
4.4.2 Modelo de permisos en Android **142**
4.4.3 Asegurando la capa de aplicación **143**
4.4.4 Evitar almacenar datos confidenciales en el dispositivo **144**
4.4.5 Uso adecuado del componente WebView **145**
4.4.6 Usar método POST para el envío de datos confidenciales **146**
4.4.7 Validar los certificados SSL/TLS **147**
4.4.8 Restricción de uso de la aplicación a determinados dispositivos **147**
4.4.9 Gestión de logs **148**
4.4.10 Comprobar la conexión de red **149**
4.4.11 Realizar operaciones de red en un hilo separado **149**
4.4.12 Permisos de localización **151**
4.4.13 Optimizar el código en Android y memoria caché **151**
4.4.14 Implementación segura de proveedores de contenido **153**
4.4.15 Almacenamiento de preferencias compartidas (SharedPreferences) **154**
4.4.16 Almacenamiento seguro de preferencias **157**
4.4.17 Almacenamiento en ficheros **160**
4.4.18 Almacenamiento externo **160**
4.4.19 Implementación segura de Intents **162**
4.4.20 Implementación segura de servicios **163**
4.4.21 Implementación segura de broadcast receivers **163**
4.4.22 Implementación segura de content providers **164**
4.4.23 Invocar actividades de forma segura **164**
4.4.24 Implementar almacenamiento de datos seguro **165**
4.4.25 Algoritmos criptográficos **170**
4.4.26 Uso de java.util.String para almacenar información sensible **172**
4.4.27 Proteger la configuración de la aplicación **172**
4.4.28 Cifrado en base de datos SQLite **173**
4.4.29 Optimización y ofuscación del código con ProGuard **174**
4.5 Metodología OASAM **177**

Capítulo 5. Seguridad en proyectos NodeJS **179**

5.1 Introducción a NodeJS **179**
5.2 Modelo Event-Loop **181**
5.3 Gestión de paquetes **182**
5.4 Programación asíncrona **184**
5.5 Problema del código piramidal **185**
5.6 Módulo para administrar el sistema de archivos **186**
5.7 Módulo http **188**
5.8 Utilización del Middleware Express **190**
5.8.1 Middleware de nivel de aplicación **190**
5.8.2 Middleware de nivel de direccionamiento **191**
5.8.3 Middleware de terceros **191**
5.9 Autenticación en NodeJS **192**
5.9.1 Auth0 . . . . . **192**
5.9.2 PassportJS **192**

5.10 OWASP top 10 en NodeJS **193**
5.10.1 OWASP NodeGOAT. **193**
5.10.2 Inyección de código **196**
5.10.3 Función eval. **197**
5.10.4 Ataque de denegación de servicio **198**
5.10.5 Uso de patrones y expresiones regulares **198**
5.10.6 Acceso al sistema de ficheros **200**
5.10.7 Inyección de SQL **202**
5.10.8 Inyección de NoSQL. **203**
5.10.9 Inyección de logs **205**
5.10.10 Gestión de la sesión y autenticación **205**
5.10.11 Protegiendo credenciales de usuario. **206**
5.10.12 Tiempo de espera de sesión y protección de cookies. **208**
5.10.13 Secuestro de sesión (Session hijacking). **212**
5.10.14 Módulo helmet. **213**
5.10.15 Cross-site scripting (XSS). **216**
5.10.16 Referencias de objetos directos inseguros **218**
5.10.17 Mala configuración de seguridad. **219**
5.10.18 Deshabilitar fingerprinting. **221**
5.10.19 Exposición de datos sensibles. **222**
5.10.20 Configurando SSL/TLS. **222**
5.10.21 Forzar peticiones HTTPS. **225**
5.10.22 Falta de control de acceso. **227**
5.10.23 Redirecciones no validadas **228**
5.10.24 Denegación de servicio mediante expresiones regulares **229**
5.10.25 Validar datos de entrada con validator **230**
5.10.26 Validar datos de entrada con express-validator. **231**
5.10.27 Configuración de cabeceras HTTP **233**
5.10.28 Política de seguridad de contenido (CSP) **235**
5.10.29 Cross-site request forgery (CSRF). **236**
5.10.30 Ejecutar código JavaScript de forma aislada **238**
5.10.31 Uso de componentes con vulnerabilidades conocidas. **239**
5.10.32 NodeJsScan. **242**

Capítulo 6. Seguridad en proyectos Python **245**

6.1 Componentes inseguros en Python. **248**
6.2 Validación incorrecta de entrada/salida. **249**
6.3 Función eval() **250**
6.4 Serialización y deserialización de datos con pickle. **253**
6.5 Ataques de inyección de entrada. **257**
6.5.1 Inyección de comandos **258**
6.5.2 Inyección de SQL **264**
6.6 Acceso seguro al sistema de archivos y ficheros temporales **266**
6.7 Inyección de XSS **268**
6.8 Inyección de SSTI **270**
6.9 Servicios para comprobar la seguridad de proyectos Python **273**
6.9.1 Pyup **273**
6.9.2 LGTM en proyectos Python. **275**

6.9.3 Sanitización de las URL **275**
6.9.4 Uso de un algoritmo criptográfico roto o débil. **276**
6.9.5 Peticiones con requests sin validación de certificado **276**
6.9.6 Uso de la versión insegura SSL/TLS **277**
6.9.7 Deserialización de entrada no confiable **277**
6.9.8 Vulnerabilidades de XSS **278**
6.9.9 Exposición de información a través de una excepción **279**
6.9.10 Conexión con hosts remotos mediante SSH utilizando Paramiko **280**
6.10 Análisis estático de código Python **280**
6.10.1 Python Taint **281**
6.10.2 Bandit **282**
6.10.3 Hawkeye **286**
6.10.4 DLint **288**
6.11 Gestión de dependencias **289**
6.11.1 Instalación de dependencias **290**
6.11.2 Requires.io **291**
6.11.3 Safety. **292**
6.11.4 Paquetes maliciosos en PyPI. **292**
6.12 Python code checkers **293**
6.12.1 Pyflakes. **293**
6.12.2 PyLint **294**
6.13 Escáner de seguridad de aplicaciones web **295**
6.13.1 WAScan. **295**
6.13.2 SQLmap. **296**
6.13.3 XSScrapy **297**
6.14 Seguridad en Django. **298**
6.14.1 Protección ante ataques XSS. **299**
6.14.2 Protección ante ataques CSRF **299**
6.14.3 Protección de inyección de SQL **300**
6.14.4 Protección de clickjacking **301**
6.14.5 SSL/HTTPS **301**
6.15 Otras herramientas de seguridad **302**
6.15.1 Yosai **302**
6.15.2 Flask-Security. **303**
6.15.3 OWASP Python Security Project **304**

Capítulo 7. Análisis estático y dinámico en aplicaciones C/C++ **305**

7.1 Análisis estático. **306**
7.1.1 Code Analyzer **307**
7.2 Análisis estático de código C/C++ **308**
7.2.1 Flawfinder **308**
7.2.2 Clang **312**
7.2.3 Uso de variables sin inicializar **312**
7.2.4 Uso inseguro de funciones **314**
7.2.5 RATS **315**
7.2.6 Vulnerabilidad cadena de formato (format string) **315**
7.2.7 Pscan para detectar vulnerabilidades format string **316**
7.2.8 Buffer overflow **317**

7.2.9 Tipos de heap overflow **320**
7.2.10 Vulnerabilidad use after free **321**
7.2.11 Dereference after free **322**
7.2.12 Vulnerabilidad double free **323**
7.2.13 Vulnerabilidad off by one **324**
7.2.14 Vulnerabilidades race condition **325**
7.2.15 Vulnerabilidad integer overflow **327**
7.2.16 Uso de StackOverflow **328**
7.3. Análisis dinámico **329**
7.3.1. Análisis dinámico en C/C++ con Valgrind **330**
7.4 Herramientas de análisis **333**

Capítulo 8. Metodologías de desarrollo **338**

8.1. Metodologías de desarrollo de software seguro **338**
8.1.1 Correctness by Construction (CbyC) **339**
8.1.2 Security Development Lifecycle (SDLC) **341**
8.1.3 Fases de la metodología SDLC **342**
8.1.4 Vulnerabilidades en SDLC **346**
8.1.5 Tipos de SDLC **349**
8.1.5.1 Microsoft Trustworthy Computing SDL **349**
8.1.5.2 CLASP **350**
8.1.5.3 TSP-Secure **351**
8.1.5.4 Oracle Software Security Assurance **352**
8.1.5.5 Propuesta híbrida **352**
8.1.6 Tipos de pruebas de seguridad SDLC **353**
8.1.7 Conclusiones de ciclo de vida de desarrollo de software (SDLC) **355**
8.2 Modelado de amenazas **356**
8.2.1 Modelado de amenazas con STRIDE **358**
8.3 Perspectiva del atacante **361**
8.4 Patrones de ataque **362**
8.5 OWASP Testing Framework y perfiles para pruebas de seguridad **363**
8.6 OWASP Security Knowledge Framework (SKF) **365**
8.7 Seguridad en ingeniería del software **374**
8.8 Bibliografía y fuentes de información **376**
8.9 Conclusiones **378**

Capítulo 9. Glosario de términos **380**

Capítulo 1. Introducción al desarrollo seguro

La necesidad de desarrollar aplicaciones seguras y, por tanto, tener en cuenta la seguridad en las metodologías de desarrollo es tan importante como tenerla en cuenta para la construcción de edificios o barcos. Desarrollar aplicaciones sin considerar la seguridad es como fabricar un barco sin botes salvavidas o sin los botes necesarios para salvar a todas las personas a bordo en caso de una catástrofe como la del Titanic.

Actualmente, las nuevas tecnologías son un orquestador fundamental en la sociedad de la información, lo cual incluye a las personas y los nuevos tipos de relaciones que se establecen entre ellas. Esta dependencia se halla presente en múltiples escenarios y contextos, desde actividades comerciales y sociales hasta el ámbito militar o la gestión de las infraestructuras críticas nacionales. Ello ha provocado la necesidad creciente de que la tecnología se comporte como se espera, esto es, de acuerdo con sus especificaciones, y de que implemente los mecanismos de seguridad apropiados desde el punto de vista de la confidencialidad, privacidad y seguridad de los datos.

Es cierto que algunos escenarios demandan unos requisitos de seguridad más rigurosos que otros. Sin embargo, la interdependencia entre sistemas de información en un contexto global como el actual nos sugiere la necesidad de una aproximación global, lo que hace de la confiabilidad en la tecnología un concepto de aplicación universal y no particular a un campo específico.

A pesar de ello, se ha demostrado ampliamente que la tecnología actual no ha alcanzado el grado de madurez y seguridad requerido, pues es vulnerable a una gran cantidad de amenazas. Según estudios realizados, cerca del 90 % de los incidentes de seguridad del software están causados por atacantes que han explotado fallos conocidos en él. Además, en un análisis de 45 aplicaciones de negocio electrónico, se mostró que el 70 % de los fallos del software estaban relacionados con el diseño. De media, un ingeniero de software experimentado y capacitado introduce un fallo por cada nueve líneas de código. Aproximadamente, un millón de líneas de código podrían tener de mil a cinco mil fallos, los cuales podrían ser el origen de alrededor de cien vulnerabilidades en producción. Además, se ha demostrado que el software se degrada con el tiempo, y lo que vale en un momento en concreto puede no valer al cabo de seis meses.

En este sentido, la ingeniería de software de desarrollo seguro es una disciplina de la ingeniería del software en la que se busca proporcionar garantías objetivas respecto a la seguridad del software desarrollado. Para ello, se aplican

mecanismos y se producen evidencias durante el proceso que garantizan que el software contiene las propiedades de seguridad que se le requieran.

En particular, y partiendo del axioma "la seguridad absoluta no es alcanzable", el software seguro es capaz de minimizar el impacto de la mayoría de los ataques, tolerar aquellos que no pueda resistir, y recuperarse rápidamente y con el menor impacto de aquellos otros que no pueda tolerar.

El campo de la investigación en ingeniería de software seguro se centra en desarrollar métodos eficaces y herramientas que permitan al desarrollador, durante el ciclo de vida del desarrollo de software, la implementación segura desde las primeras fases de análisis hasta las etapas de más largo plazo de desarrollo.

Uno de los grandes retos actuales para la aplicación de cualquier técnica orientada a mejorar la calidad y la seguridad del software es el cambio de mentalidad en el desarrollador, así como, en muchos casos, la compleja integración de estas técnicas en los procesos existentes, con los costes que ello conlleva. Los hechos nos demuestran que resulta urgente mejorar los procesos de ingeniería del software en pro de unos sistemas de información más robustos y seguros que aporten la confianza necesaria a la sociedad.

El desarrollo seguro es una parte importante de la seguridad informática, englobado dentro del ámbito de la prevención. Podríamos definir que un programa seguro es aquel capaz de seguir realizando las funciones para las que ha sido creado en todo momento, y capaz de evitar que la existencia de errores en él pueda ser utilizada como puente para la realización de acciones que pongan en peligro la integridad, confidencialidad o disponibilidad del resto de elementos del sistema en el que se está ejecutando.

Por tanto, la programación segura engloba el conjunto de técnicas, normas y conocimientos que permiten crear programas que no puedan ser modificados de forma ilegítima con fines maliciosos y que estén carentes de fallos que puedan comprometer la seguridad del resto de elementos del sistema con el que interactúan.

Los objetivos de la utilización de prácticas de software seguras son las siguientes:

- Los fallos explotables y otros puntos débiles se eliminan en la mayor medida posible.
- La probabilidad de que los desarrolladores puedan producir errores y vulnerabilidades explotables o puertas traseras en el software se reduce o elimina en gran medida.

- El software es resistente y tolerante a posibles ataques para apoyar el cumplimiento de la misión de la organización.

Uno de los principales objetivos radica en ayudar a desarrolladores de software a implementar medidas preventivas que ayuden a mitigar o reducir lo máximo posible errores comunes de programación y que pueden ser aprovechados por un posible atacante para comprometer una aplicación.

El libro presenta gran interés para cualquier desarrollador responsable de una página web o un administrador de grandes portales corporativos, así como para analistas y programadores de una empresa de las denominadas *software factories*. El lector estará en disposición de definir procedimientos y políticas de desarrollo de código dentro de una misma organización, y también de establecer los controles necesarios para el desarrollo de software seguro.

1.1 Propiedades del software seguro

Si bien con las metodologías y los estándares relacionados con el desarrollo y la construcción de software se busca mantener altos niveles de confiabilidad y control de la solución informática, la seguridad informática y sus principios de diseño seguro no suelen ser parte formal u obligatoria de dichos estándares.

Cuando una aplicación transmite o almacena información confidencial, la aplicación es responsable de garantizar que los datos almacenados y transferidos estén cifrados y no puedan obtenerse, alterarse o divulgarse fácilmente de forma ilícita. En general, se suele decir que el objetivo fundamental de la seguridad informática se basa en preservar los siguientes puntos:

- **Confidencialidad**. El software debe asegurar que cualquiera de sus características, los activos que administra y su contenido son accesibles solo para las entidades autorizadas e inaccesibles para el resto. El acceso a la información está limitado a usuarios autorizados. Los datos deben protegerse de la observación o divulgación no autorizadas tanto en tránsito como almacenadas.
- **Integridad**. El software y los activos del sistema solo pueden ser modificados por usuarios autorizados. Esta propiedad se debe preservar durante el desarrollo del software y su ejecución. Los datos han de protegerse al ser creados, alterados o borrados maliciosamente por atacantes no autorizados.

- **Disponibilidad**. El software debe estar operativo y ser accesible a sus usuarios autorizados siempre que se requiera, así como ha de desempeñarse con una *performance* adecuada para que los usuarios puedan realizar sus tareas de forma correcta y dar cumplimiento a los objetivos de la organización que lo utiliza. El acceso a los activos en un tiempo razonable está garantizado para usuarios autorizados. Los datos deben encontrarse disponibles para los usuarios autorizados, según sea necesario.

Para las entidades que actúan como usuarios, se requieren dos propiedades adicionales:

- **Trazabilidad**. Todas las acciones relevantes relacionadas con la seguridad de una entidad que actúa como usuario se deben registrar y trazar con el objetivo de disponer de datos para la realización de auditorías; de esta forma, la trazabilidad debe ser posible tanto durante la ocurrencia de las acciones registradas como *a posteriori*.
- **No repudio**. Constituye la habilidad de prevenir que una entidad que actúa como usuario desmienta o niegue la responsabilidad de acciones que han sido ejecutadas.

Estas propiedades básicas son las más utilizadas normalmente para describir la seguridad de los sistemas y aplicaciones. Un ataque con éxito de **inyección de SQL** en una aplicación para extraer información de identificación personal de su base de datos sería una violación de la propiedad de **confidencialidad**. El éxito de un ataque **cross-site scripting** (XSS) en contra de una aplicación web podría dar lugar a una violación tanto en su **integridad** como en su **disponibilidad**. Un ataque con éxito de desbordamiento de búfer que inyecta código con el objetivo de obtener y modificar la información de usuarios sería una violación de las cinco propiedades básicas de seguridad.

1.2 Principios de diseño seguro de aplicaciones

En consecuencia, los efectos de vulnerar la seguridad del software se pueden describir en términos de los efectos sobre estas propiedades fundamentales. A continuación se sugieren una serie de principios orientados al diseño seguro de aplicaciones:

1.2.1 Minimizar el área de la superficie de ataque

Cada característica que se añade a una aplicación incrementa su complejidad y aumenta también el riesgo de aplicación en conjunto. Una nueva característica implica un nuevo punto de ataque. Uno de los factores clave para reducir el riesgo de una aplicación recae en la reducción de la superficie de ataque. Pueden eliminarse posibles puntos de ataque si se deshabilitan módulos o componentes innecesarios para la aplicación; por ejemplo, si la aplicación no utiliza el almacenamiento en caché de resultados, sería recomendable deshabilitar dicho módulo. De esta manera, si se detecta una vulnerabilidad de seguridad en ese módulo, la aplicación no se verá amenazada.

1.2.2 Seguridad por defecto

Existen muchas maneras de entregar una experiencia *out of the box* ("lista para usar") a los usuarios. Sin embargo, por defecto, la experiencia debe ser segura, mas facilitar, no obstante, la capacidad de reducción del nivel de seguridad si el usuario lo cree necesario.

Del lado de los desarrolladores, constituye una práctica habitual utilizar opciones de configuración de seguridad reducidas para evitar que dichas configuraciones compliquen el desarrollo. Si, para su implementación, la aplicación requiere de características que obligan a reducir o cambiar la configuración de seguridad predeterminada, se recomienda estudiar sus efectos y consecuencias, para lo que hay que realizar pruebas específicas de auditoría de seguridad.

1.2.3 Privilegios mínimos

Según el principio del mínimo privilegio, se recomienda que las cuentas de usuario tengan la mínima cantidad de privilegios necesarios. Asimismo, se aconseja que los procesos se ejecuten únicamente con los privilegios necesarios para completar sus tareas. De esta manera, se limitan los posibles daños que podrían producirse si se ve comprometido el proceso.

Si un atacante llegase a tomar el control de un proceso en un servidor o comprometiese una cuenta de usuario, los privilegios concedidos determinarán, en gran medida, los tipos de operaciones que podrá llegar a realizar dicho atacante; por ejemplo, si cierto servidor solo requiere acceso de lectura a la tabla de una base de datos, bajo ninguna condición deberían darse privilegios administrativos a los usuarios si no resultan necesarios.

1.2.4 Validación de datos de entrada

Se ha de garantizar que la aplicación sea robusta ante todas las posibles formas de entrada de datos, ya sean proporcionados por el usuario, por la infraestructura, por entidades externas o por bases de datos.

Una premisa fundamental reside en no confiar en los datos que el usuario pueda introducir, ya que este tiene todas las posibilidades de manipularlos. La debilidad de seguridad más común en aplicaciones es la falta de validación apropiada de las entradas del usuario o del entorno. Esta debilidad origina casi todas las principales vulnerabilidades en las aplicaciones, tales como inyecciones de código, inserción de secuencias de comandos, ataques al sistema de archivos o desbordamientos de memoria.

Las aplicaciones deben validar todos los datos introducidos por el usuario antes de realizar cualquier operación con ellos. La validación podría incluir el filtrado de caracteres especiales o el control de la longitud de los datos introducidos. Esta medida preventiva protege a la aplicación de usos incorrectos accidentales o de ataques deliberados por parte de usuarios.

1.2.5 Defensa en profundidad

Con "defensa en profundidad", nos referimos a definir una estrategia de seguridad estándar en la que se establezcan varios controles de defensa en cada una de las capas y los ssubsistemas de la aplicación. Estos puntos de control ayudan a garantizar que solo los usuarios autenticados y autorizados puedan obtener el acceso a la siguiente capa y a sus datos.

1.2.6 Control seguro de errores

Es necesario controlar las respuestas cuando se produce algún error y no mostrar en ellas información que pudiera ayudar al atacante a descubrir datos acerca de cómo ha sido desarrollada la aplicación o cómo funcionan sus procedimientos. La información detallada de los errores producidos no debería ser mostrada al usuario, sino que tendría que ser enviada al fichero de log correspondiente. Una simple página de error 403 indicando un acceso denegado puede decir a un escáner de vulnerabilidades que un directorio existe y puede proporcionar a un atacante información que le permita realizar un mapa de la estructura de directorios de la aplicación.

1.2.7 Separación de funciones

Un punto importante consiste en la separación de funciones entre los distintos perfiles de la aplicación; por ejemplo, en una tienda de subastas online, un usuario vendedor pone en subasta un producto. Si la separación de funciones se encuentra correctamente implementada, este mismo usuario no debe poder participar en la puja por el artículo. Por otra parte, determinados roles deben contar con un nivel de confianza más elevado que los usuarios normales, como en el caso del administrador, que posee privilegios avanzados, como administrar al resto de usuarios del sistema o configurar políticas de contraseñas, así como definir los diferentes parámetros de la aplicación.

1.2.8 Evitar la seguridad por oscuridad

La seguridad de una aplicación no debería depender del secreto o confidencialidad de su diseño o implementación. Si se intentan ocultar secretos mediante el uso de nombres de variables engañosos o de ubicaciones de archivos no habituales, no se estará mejorando la seguridad. La seguridad basada en la oscuridad es un control de seguridad débil, especialmente si se trata del único control. Esto no significa que mantener secretos constituya una mala idea; significa que la seguridad de los sistemas clave no debería basarse, exclusivamente, en mantener detalles ocultos.

Por ejemplo, la seguridad de una aplicación no debería basarse en mantener en secreto el conocimiento del código fuente. La seguridad ha de fundamentarse en muchos otros factores, incluyendo políticas de contraseñas, diseño de una arquitectura sólida tanto a nivel de aplicación como a nivel de comunicaciones de red y realización periódica de controles de auditoría. Un ejemplo práctico es Linux, cuyo código fuente es open source y está disponible para todo el mundo y, aun así, se trata de un sistema operativo seguro y robusto.

1.3 Análisis de requisitos de seguridad

La seguridad debería ser un aspecto de alta prioridad cuando estamos desarrollando y probando aplicaciones que manejan datos confidenciales a nivel de autenticación y autorización. El análisis de los requisitos de seguridad debe ser parte de la primera fase de análisis de los requisitos de cada proyecto de aplicaciones móviles. La siguiente lista lo puede ayudar con el análisis de los requisitos de seguridad:

- Identificar los posibles roles de usuario, así como sus limitaciones y permisos dentro de la arquitectura (app y backend).
- Identificar qué tipo de enfoques y herramientas de pruebas de seguridad se requieren para lograr un buen nivel de seguridad.
- Identificar el impacto que tienen las funcionalidades a nivel de usuario en la seguridad a nivel frontend y backend.

La mayoría de los fallos de seguridad de aplicaciones se pueden prevenir mediante la integración de los procesos y buenas prácticas de seguridad desde las primeras etapas de desarrollo. La planificación de una estrategia inicial de diseño de aplicaciones y mantenimiento de la seguridad en mente todo el tiempo permite reducir las posibilidades de los riesgos de seguridad que surgen durante las últimas etapas de desarrollo de aplicaciones.

Capítulo 2. Aspectos fundamentales de desarrollo seguro

Resulta importante hacer entender que la seguridad debe formar parte del ciclo de desarrollo como algo integrado dentro de las herramientas de integración continua, no como una fase separada, sino como parte integral del desarrollo. El objetivo, al final, es tratar de minimizar los riesgos que pueda suponer la seguridad de las aplicaciones.

Una buena práctica sería poner toda la lógica de negocio posible en la parte de servidor/backend, para evitar que alguien pueda hacer ingeniería inversa en la aplicación y averiguar determinados datos. En el caso de que haya que introducir datos sensibles, se deberían encriptar para evitar ataques.

Cuando se habla de seguridad, los atacantes tienen una ventaja sobre los desarrolladores. Esta se expone en cuatro principios que la ilustran muy bien:

- Quien defiende debe preocuparse por proteger todos los puntos; el atacante seleccionará solo uno: el más débil.
- Los encargados de la defensa van a procurar defenderse de ataques conocidos; el atacante probará nuevas formas de llevar adelante sus ataques.
- Quien se encarga de defender debe estar en constante estado de vigilancia.
- El defensor ha de respetar reglas; el atacante, normalmente, juega con sus propias reglas.

Cabe resaltar que un atacante necesita solamente un pequeño error, una vulnerabilidad, para lograr su cometido. Si dentro de la empresa se descuidan los temas de seguridad por acelerar la operatividad del negocio, podemos estar dejando la puerta abierta a que se comprometa la seguridad de la información. Ser responsables con los procesos constituye la mejor defensa, y no está de más preguntarse si es mejor invertir unas semanas más en desarrollo que perder reputación y dinero en un instante por un incidente de seguridad.

Controlar un sistema resulta parte fundamental para garantizar un mejoramiento continuo. Por esta razón, vamos a ver algunos controles que deberían tenerse en cuenta para que una aplicación garantice la integridad y disponibilidad de la información y, además, se pueda mejorar progresivamente.

En particular, para aquellos que están directamente relacionados con el desarrollo, la responsabilidad es mayor, pues, finalmente, son quienes atienden los requerimientos e implementan las soluciones.

Por esta razón, se recomienda tener en cuenta algunos controles en sus desarrollos, para prevenir escenarios como el **cross-site scripting (XSS)**, las inyecciones de código, la ejecución de códigos maliciosos y el **cross-site request forgery**. Posteriormente, revisamos el proyecto de OWASP, que publica un top 10 de vulnerabilidades, y donde se presentan los riesgos más críticos de aplicaciones web, junto con la forma de prevenirlos.

Los controles implementados dentro de los desarrollos deben validar la entrada, el procesamiento y la salida de los datos, de forma que se cumpla con los niveles de confidencialidad, integridad y disponibilidad.

De esta forma, deben implementarse **controles de detección**, que ayudan a identificar los accesos no autorizados o entradas no válidas; **controles preventivos**, diseñados para evitar la entrada de datos no autorizados o no válidos, y **controles proactivos**, que permiten probar los componentes de forma continua, a medida que se desarrollan, mediante pruebas unitarias y de integración.

Los **controles de detección** aseguran que las operaciones se hagan con la exactitud requerida y, además, que las consultas que se realicen sobre bases de datos u otros sistemas sean consistentes. Operaciones como la combinación de archivos, la modificación de datos, la actualización de bases de datos o los mantenimientos de sistemas son operaciones sensibles que deberían tener este tipo de controles.

Los **controles preventivos** deben enfocarse en que la información ingresada sea precisa y, por lo tanto, en que aquello que se registre en el sistema sea adecuado. Dentro de estos controles, se pueden tener validaciones mediante el uso de expresiones regulares, comprobar la integridad de los campos, verificar que no exista duplicidad en los datos, establecer rangos de validación e incluso comprobar que los valores de los campos sean razonables de acuerdo con unos criterios predefinidos.

2.1 Controles proactivos

Independientemente de los controles implementados, resulta muy importante, además, definir un adecuado plan de pruebas que incluya, asimismo, las pruebas de aceptación por parte de los usuarios finales, donde se evalúa la funcionalidad de la aplicación.

Aparte de los usuarios finales, deberían considerarse recursos del equipo de desarrollo para realizar pruebas de funcionalidad del código y de integración.

Incluso si la aplicación es muy crítica para el negocio, se podría considerar la alternativa de que un equipo externo al equipo de desarrollo también complemente con otras pruebas a las ya realizadas por el equipo de desarrollo.

Para lograr un software seguro, los desarrolladores deben contar con el respaldo y la ayuda de la organización para la que escriben el código, cosa que, a veces, no sucede. A medida que los desarrolladores de software crean el código que compone una aplicación, se necesita adoptar y practicar una amplia variedad de técnicas de codificación segura. Todas las capas que componen una aplicación, tales como la interfaz de usuario, la lógica de negocio, el controlador o la base de datos, deben desarrollarse teniendo la seguridad en mente. La mayoría de los desarrolladores no han aprendido sobre la codificación segura o la criptografía en su etapa de formación y, en ocasiones, tampoco se lo han exigido.

Los lenguajes y frameworks que los desarrolladores usan para construir aplicaciones a menudo carecen de controles centrales críticos o son inseguros por defecto de alguna manera. También, en honor a la verdad, es muy raro que las organizaciones brinden a los desarrolladores requisitos prescriptivos que los guíen por el camino del software seguro e, incluso, cuando lo hacen, puede haber errores de seguridad inherentes a los requisitos y diseños. Por ello, se ha de intentar ser proactivo desde los inicios del diseño del proyecto que se realice, teniendo en cuenta, en principio, la seguridad.

2.2 OWASP (Open Web Application Security Project)

OWASP (Open Web Application Security Project) nos provee de una serie de recursos basados, sobre todo, en guías y herramientas, para que nuestros proyectos web sean lo más seguros posible, tanto desde el punto de vista de desarrollo seguro como de la evaluación de la seguridad de estos.

Para los desarrolladores y auditores de seguridad, ofrece una gran cantidad de recursos para ayudar a llevar a cabo un ciclo de vida de desarrollo de software seguro (S-SDLC, Secure Software Development Life Cycle), así como una variedad de recursos, como guías y herramientas, para evaluar la seguridad de las aplicaciones web.

Entre los principales objetivos de la OWASP, podemos destacar:

- Proporcionar a los desarrolladores un conjunto de buenas prácticas, para que las aplicaciones sean lo más seguras posible.

- Proporcionar a desarrolladores y profesionales de seguridad recursos para asegurar aplicaciones; en concreto, para aplicaciones móviles, surgió el proyecto OWASP Mobile Security Project.

El objetivo del proyecto es intentar ayudar a los desarrolladores a comprender qué, por qué, cuándo, dónde y cómo probar aplicaciones web. Los desarrolladores pueden usar este framework como plantilla para construir sus propios programas de prueba o para evaluar los procesos de otros desarrolladores. En la guía se describen en detalle tanto el framework de pruebas en general como las técnicas requeridas para llevarlo a la práctica.

El proyecto de seguridad en aplicaciones web (OWASP) es una comunidad abierta dedicada a facultar a las organizaciones a desarrollar, adquirir y mantener aplicaciones que pueden resultar confiables. Esta organización nos presenta los 10 riesgos más comunes, entre los que podemos destacar:

- **Inyección**. Los errores de inyección, tales como SQL y LDAP, ocurren cuando datos no confiables son enviados a un intérprete como parte de un comando o consulta. Los datos hostiles del atacante pueden engañar al intérprete a la hora de ejecutar comandos no intencionados o acceder a datos no autorizados.
- **Pérdida de autenticación y gestión de sesiones**. Las funciones de la aplicación relacionadas con la autenticación y gestión de sesiones son, frecuentemente, implementadas de manera incorrecta, lo que permite a los atacantes comprometer contraseñas, claves, *token* de sesiones o explotar otros errores de implementación para asumir la identidad de otros usuarios.
- Secuencia de comandos en sitios cruzados (XSS). **Los errores XSS ocurren** cada vez que una aplicación toma datos no confiables y los envía al navegador web sin una validación y codificación apropiada. XSS permite a los atacantes ejecutar una secuencia de comandos en el navegador de la víctima, los cuales pueden secuestrar las sesiones de usuario, destruir sitios web o dirigir al usuario hacia un sitio malicioso.
- **Referencia directa insegura a objetos**. Una referencia directa a objetos ocurre cuando un desarrollador expone una referencia a un objeto de implementación interno, tal como un fichero, directorio o base de datos. Sin un chequeo de control de acceso u otra protección, los atacantes pueden manipular estas referencias para acceder a datos no autorizados.

- **Configuración de seguridad incorrecta**. Una buena seguridad requiere tener definida e implementada una configuración segura para la aplicación, marcos de trabajo, servidor de aplicación, servidor web, base de datos y plataforma. Todas estas configuraciones deben ser definidas, implementadas y mantenidas ya que, por lo general, no son seguras por defecto. Esto incluye mantener todo el software actualizado, incluidas las librerías de código utilizadas por la aplicación.
- **Exposición de datos sensibles**. Muchas aplicaciones web no protegen adecuadamente datos sensibles, tales como números de tarjetas de crédito o credenciales de autenticación. Los atacantes pueden robar o modificar tales datos para llevar a cabo fraudes, robos de identidad u otros delitos. Los datos sensibles requieren de métodos de protección adicionales, tales como el cifrado de datos, así como también de precauciones especiales en un intercambio de datos con el navegador.
- **Ausencia de control de acceso a funciones**. La mayoría de las aplicaciones web verifican los derechos de acceso a nivel de función antes de hacer visible una funcionalidad en la interfaz de usuario. A pesar de esto, las aplicaciones necesitan verificar el control de acceso en el servidor cuando se accede a cada función. Si las solicitudes de acceso no se verifican, los atacantes podrán realizar peticiones sin la autorización apropiada.
- **Falsificación de peticiones en sitios cruzados (CSRF)**. Un ataque CSRF obliga al navegador de una víctima autenticada a enviar una petición HTTP falsificada, incluyendo la sesión del usuario y cualquier otra información de autenticación, incluida automáticamente, a una aplicación web vulnerable. Esto permite al atacante forzar al navegador de la víctima para generar peticiones que la aplicación vulnerable piensa que son peticiones legítimas.
- **Utilización de componentes con vulnerabilidades conocidas**. Algunos componentes tales como las librerías, los frameworks y otros módulos de software casi siempre funcionan con todos los permisos y privilegios. Si se ataca un componente vulnerable, esto podría facilitar la intrusión en el servidor o una pérdida de datos. Las aplicaciones en las que se utilicen componentes con vulnerabilidades conocidas permiten ampliar el rango de posibles ataques.
- **Redirecciones y reenvíos no validados**. Las aplicaciones web, frecuentemente, redirigen y reenvían a los usuarios hacia otras páginas o

sitios web y utilizan datos no confiables para determinar la página de destino. Sin una validación apropiada, los atacantes pueden redirigir a las víctimas hacia sitios de phishing o malware, o utilizar reenvíos para acceder a páginas no autorizadas.

Un proyecto interesante que ofrece OWASP es la aplicación open source **Zed Attack Proxy Project (ZAP)** https://www.zaproxy.org, que nos permite realizar un análisis de todos los datos que se envían y que recibimos a la hora de realizar una navegación de un sitio web. La herramienta se puede descargar desde el repositorio de github: https://github.com/zaproxy/zaproxy/wiki/Downloads

La herramienta soporta análisis automático y manual. El primero es más sencillo y menos eficiente, ya que, si se requiere introducir login en la web, solo va a escanear hasta ese punto. Para utilizar el modo automático, bastaría con introducir la url que deseamos analizar en el apartado "url a atacar" y presionar el botón "Atacar".



Figura 2.1 Modos de funcionamiento de OWASP ZAP.

En el modo manual, ZAP actúa como un proxy de intercepción de las peticiones que se realizan en el navegador, de forma que es capaz de analizar cada una de las url para extraer valores de formularios y parámetros de navegación del usuario. Utilizando esta opción, OWASP ZAP utiliza un proxy propio para capturar las peticiones que realizamos durante una sesión de navegación. Toda la navegación efectuada será monitorizada en la aplicación ZAP y se irá mostrando en el apartado "Sitios". En el panel inferior se ven los resultados obtenidos en forma de alertas.



Figura 2.2 Alertas obtenidas al realizar un análisis sobre un sitio web.

2.3. OWASP Mobile Security Project

El objetivo del proyecto reside en crear conciencia acerca de la seguridad en aplicaciones, identificando algunos de los riesgos más críticos a los que se enfrentan las organizaciones. Pues bien, tanto los desarrolladores de aplicaciones móviles como los auditores pueden aprovecharse también de recursos de este tipo para alcanzar el objetivo de la máxima seguridad posible. Para ello, OWASP nos propone el proyecto Mobile Security Project.

La checklist de OWASP para dispositivos móviles está centrada en la seguridad referente a aplicaciones móviles. Trata de dar a los desarrolladores de aplicaciones un recurso centralizado para mantener las aplicaciones seguras. El objetivo es clasificar los riesgos de seguridad y proveer de controles para reducir el impacto o el riesgo de explotación. OWASP se focaliza en la capa de aplicación, en la aplicación entre la aplicación y los recursos remotos de servicios de autenticación, y en las características específicas de plataformas *cloud*.

El proyecto de seguridad móvil OWASP ofrece a los desarrolladores y equipos de seguridad los recursos para construir y mantener aplicaciones móviles seguras. También permite clasificar los riesgos de seguridad móvil y proporciona controles de desarrollo para reducir su impacto o probabilidad de explotación.

Tal y como podemos encontrar en el top 10 de riesgos en aplicaciones móviles https://www.owasp.org/index.php/Mobile_Top_10_2016-Top_10, la siguiente lista nos indica el top 10 de los controles de seguridad para evaluar en aplicaciones móviles:

- **M1-Improper Platform Usage (uso indebido de plataformas)**. Esta categoría cubre el mal uso de las características de las plataformas móviles o de los fallos de uso en los controles de seguridad. Puede incluir permisos en las plataformas, así como la mala utilización de TouchID,

del llavero u otros controles de seguridad que forman parte del sistema operativo.

- **M2-Insecure Data Storage (almacenamiento de datos inseguro)**. Esta categoría cubre las antiguas categorías M2 y M4 del año 2014 (M2: Weak Server Side Controls; M4: Client Side Injection). Cubre el almacenamiento de datos inseguro y la fuga de datos no deseados.
- **M3-Insecure Communication (comunicaciones inseguras).** Esta categoría cubre un mal establecimiento de conexión o handshake, versiones SSL inseguras o comunicaciones en texto en claro sensibles. Toda comunicación donde se transmitan datos de usuarios debe realizarse mediante un canal seguro. Si la aplicación implementa sesiones de usuario mediante el uso de cookies, se deben proteger todas las conexiones contra el servidor.
- **M4-Insecure Authentication (autenticación insegura)**. Esta categoría incluye nociones de autenticación del usuario final o una mala gestión de la sesión del usuario. Algunos ejemplos son no poder identificar al usuario, con lo que se pierde trazabilidad de las acciones, o no mantener la identidad del usuario cuando se requiere.
- **M5-Insufficient Cryptography (criptografía insuficiente)**. Se hace uso de la criptografía en el código para proteger activos sensibles. Sin embargo, a veces, la criptografía no es suficiente en las aplicaciones, ya que se emplean algoritmos criptográficamente rotos, como MD5 o SHA1. En este punto resulta recomendable usar, al menos, un hash del tipo SHA256, que devuelve un bloque de 256 ante cualquier entrada. Al aumentar los bits de salida del algoritmo, en comparación con el algoritmo MD5, disminuye la posibilidad de encontrar colisiones; por lo tanto, es más seguro.
- **M6-Insecure Authorization (autorización insegura)**. Se refiere a fallos en las autorizaciones en las aplicaciones. Es diferente a problemas de autenticación e identificación de usuarios. Cuando las aplicaciones autorizan, dan acceso a los usuarios a visualizar en la aplicación solo aquello que debe visualizar. Se ha de dotar de permisos al usuario, ya sean de escritura, lectura, ejecución u otros permisos más afines al negocio.
- **M7-Client Code Quality (calidad del código de cliente)**. Aquí podríamos incluir problemas relacionados, como desbordamiento de búfer, vulnerabilidades de cadena de formato y otros varios errores a nivel de código, donde la solución estriba en volver a escribir algo de código, que se ejecuta en el dispositivo móvil.

- **M8-Code Tampering (manipulación de código)**. Esta categoría incluye parchear binarios o modificar recursos locales y memoria dinámica. Un atacante puede directamente modificar el código, cambiar contenidos en la memoria dinámica o incluso reemplazar el sistema de API, que usa la aplicación, o bien cambiar los recursos y datos de la aplicación.
- **M9-Reverse Engineering (ingeniería inversa)**. Esta categoría incluye el análisis de binarios para obtener el código fuente y las librerías. Herramientas como IDA Pro, Hooper y otras de inspección de binarios permitirán al atacante visionar acerca del funcionamiento interno de la aplicación.
- **M10-Extraneous Functionality (funcionalidad extraña)**. En ocasiones, los desarrolladores incluyen funcionalidades ocultas de acceso mediante backdoors u otros controles de seguridad internos, no destinados al entorno de producción; por ejemplo, un desarrollador podría incluir, accidentalmente, una contraseña en un comentario de la aplicación o la posibilidad de desactivar el doble factor de autenticación durante la etapa de *testing*.

2.4 Controles proactivos OWASP

El proyecto OWASP Application Security Verification Standard (ASVS) proporciona una base para probar los controles técnicos de seguridad de aplicaciones web y también proporciona a los desarrolladores una lista de requisitos para un desarrollo seguro.

El estándar proporciona una base para probar los controles técnicos de seguridad de las aplicaciones, así como los controles técnicos de seguridad en el entorno, utilizados para proteger contra vulnerabilidades como **Cross-site scripting (XSS) e inyección de SQL**. Este estándar se puede utilizar para establecer un nivel de confianza en la seguridad de las aplicaciones web.

A continuación comentaremos los principales controles proactivos que OWASP recomienda y que deberían incluirse en cada proyecto de desarrollo de software. Entre la lista de los controles que se deben realizar, por orden de importancia, podemos destacar:

- **Verificación de la seguridad desde las primeras etapas de desarrollo**
- **Validación de las entradas del cliente**
- **Desbordamientos del búfer**
- **Gestión de sesiones**

- **Implementación de controles de acceso**
- **Implementación de controles de identidad y autenticación**
- **Autenticación por múltiples factores**
- **Manejo de errores y excepciones**

2.4.1 Verificación de la seguridad desde las primeras etapas de desarrollo

Las pruebas de seguridad deberían ser parte integral de la práctica de ingeniería de software de un desarrollador. En muchas organizaciones, esto no suele hacerse, ya que las pruebas de seguridad se realizan fuera de los ciclos de pruebas de desarrollo.

Por ello, es importante realizar comprobaciones de seguridad de manera temprana y lo más frecuentemente posible, ya sea mediante pruebas manuales o pruebas automatizadas. Obviamente, esto aumenta los plazos de entrega y lógicamente, por ello, ha de ser tenido en cuenta desde el inicio del diseño de la aplicación.

Como parte del desarrollo, debe plantearse la necesidad de implementar medidas de testing durante la fase de desarrollo. Para ello, OWASP tiene un proyecto de testing del código. Puede considerarse **OWASP ASVS (Application Security Verification Standard Project)** https://github.com/OWASP/ASVS, como una guía para definir los requisitos de seguridad y las pruebas.

2.4.2 Validación de las entradas del cliente

La debilidad más común en la seguridad de las aplicaciones web es la falta de validación adecuada de los datos que provienen del cliente o del entorno antes de utilizarlo. Se debe validar y filtrar cada input de la aplicación, tanto a nivel frontend como backend. Incluso si los datos provienen de su aplicación, son posibles su intercepción y manipulación. Sin la validación correcta de todas las entradas, podríamos exponer nuestra aplicación a ataques, como desbordamientos de búfer e inyección de SQL.

Esta debilidad conduce a casi todas las principales vulnerabilidades en las aplicaciones web, tales como scripting cross site, inyección de SQL, inyección de intérprete, ataques de sistemas de archivos y desbordamientos de búfer. En este punto cabe asegurar que la aplicación satisface los siguientes requisitos de gestión de sesiones de alto nivel:

- Todas las entradas están validadas para ser correctas y utilizables para el propósito previsto.
- Los datos de una entidad externa o cliente nunca deben ser confiables y han de tratarse en todos los casos.

La validación de las entradas se tiene que realizar tanto en el lado del cliente como en el lado del servidor; por ejemplo, la validación JavaScript puede alertar al usuario de que un campo, en particular, debe consistir en números, pero el servidor ha de validar que el campo realmente consiste en números. Por otro lado, no podemos olvidar que la validación de entrada no debe utilizarse como el método principal para prevenir vulnerabilidades del tipo XSS e inyección de SQL, pero contribuye significativamente a reducir su impacto, si se implementa correctamente.

Todos los parámetros que gestiona la aplicación han de ser validados, especialmente cuando son recibidos como datos de entrada por el usuario. De esta forma, **dicha validación se realizará tanto en la parte del cliente como a nivel de servidor**. Dentro de esta validación, se comprobará:

- Si los caracteres incluidos son correctos, no incluyen caracteres HTML o códigos SQL o JavaScript.
- Si el tipo del valor introducido es válido.
- Si su longitud se encuentra entre el valor mínimo y el máximo permitido.
- Si el rango está entre los límites permitidos.

Una gran mayoría de las vulnerabilidades de la aplicación web surgen por no validar correctamente la entrada del usuario. Los usuarios que utilizan una interfaz de usuario no necesariamente introducen esta "entrada" directamente. En el contexto de aplicaciones y servicios web, esto podría incluir los siguientes casos:

- Cabeceras HTTP
- Cookies
- Parámetros GET y POST (incluyendo campos ocultos)
- Subidas de archivos (incluyendo información, como el nombre del archivo)

Existen dos enfoques generales para realizar la validación de la sintaxis de entrada, comúnmente conocida como "lista negra" y "lista blanca":

- **La lista negra (o blacklist)** es un listado de valores no aceptables con el que se intenta comprobar que una entrada de usuario determinada no posee contenido "conocido como malicioso". Esto es similar a cómo funcionará un programa antivirus, que comprueba si un archivo coincide exactamente con el contenido malicioso conocido y, si lo hace, lo rechaza.
- **La lista blanca (o whitelist)** es un listado de valores válidos con la que se intenta comprobar que una entrada de usuario determinada coincide con un conjunto de entradas "bien conocido"; por ejemplo, una aplicación web puede permitirle seleccionar una de entre una serie de opciones: la aplicación comprobará que una de ellas ha sido seleccionada y rechazará todas las demás entradas posibles. Debido a que la validación de los datos de entrada y salida son aspectos básicos de seguridad informática, las listas blancas representan una herramienta indispensable para los desarrolladores.

Desde el punto de vista de la seguridad, siempre es mejor utilizar listas blancas. Si utilizáramos una lista negra, deberíamos introducir en esta lista todos los valores no aceptables, que podrían ser cantidades innumerables en constante crecimiento. Sin embargo, siempre sabemos qué valores son aceptables y podemos enumerarlos rápidamente, sin tener que complementar el listado en el futuro; por ejemplo, si sabemos que nuestra variable ha de tener un identificador numérico, una lista blanca incluiría tan solo valores enteros, mientras que una lista negra tendrá que incluir todo lo que no sea un valor entero (por ejemplo, letras, caracteres especiales, espacios, etc.).

Los ataques que se evitarán utilizando una correcta validación de parámetros de entrada serán:

- Cross-site scripting
- Inyección de código SQL
- Denegación de servicio en la aplicación: para evitarla, se deberán validar los tipos, las longitudes y los rangos de los parámetros de entrada, de modo que no se produzcan excepciones en la aplicación.
- Desbordamientos de búfer: mediante la validación de las longitudes de los parámetros de entrada, se evitará la ejecución de código arbitrario en el sistema.

2.4.3 Desbordamientos del búfer

Los desbordamientos del búfer son métodos que se usan para explotar el software de forma remota mediante la inyección de código malicioso en la aplicación destino. La causa raíz de los problemas de desbordamiento de búfer radica en que los lenguajes de programación más utilizados, como C y C++, tienen funciones que son inseguras. El principal problema se debe a que no se comprueban los límites de manejo de búfer, al intentar acceder a posiciones que se encuentran por encima de la zona asignada.

El comportamiento indeterminado de programas que han invadido un búfer hace que sea particularmente difícil de depurar. En el peor de los casos, un programa puede desbordar un búfer y no mostrar ningún efecto secundario adverso. Como resultado, los problemas de desbordamiento de búfer a menudo permanecen invisibles durante las pruebas. Lo más importante a la hora de analizar los desbordamientos del búfer reside en que cualquier dato que pasa a ser asignado cerca del búfer potencialmente puede ser modificado cuando se produzca el desbordamiento.

2.4.4 Gestión de sesiones

La sesión es uno de los componentes principales de cualquier aplicación basada en web, por el cual se controla y mantiene el estado de un usuario que interactúa con ella. En este punto, se ha de asegurar que la aplicación satisface los siguientes requisitos de gestión de sesiones de alto nivel:

- Las sesiones son únicas para cada individuo y no pueden ser adivinadas o compartidas.
- Las sesiones se invalidan cuando ya no son necesarias y se anulan durante los periodos de inactividad.

Las sesiones para los usuarios se mantienen, en la mayoría de las aplicaciones, a través de una cookie, que puede ser vulnerable. La mayoría de frameworks web ofrece administración de sesiones de una forma segura. También se debe asegurar que el tamaño de la cookie de sesión es suficientemente largo. Las cookies de sesión cortas o predecibles hacen posible que un atacante pueda predecir o realizar otros ataques contra la sesión.

2.4.5 Implementación de controles de acceso

El control de acceso supone el concepto de permitir el acceso a los recursos solo a quienes tienen permiso para utilizarlos. Debemos asegurarnos de que una aplicación verificada cumple los siguientes requisitos de alto nivel:

- Los usuarios que acceden a los recursos deben aportar credenciales válidas para hacerlo.
- Los usuarios están asociados con un conjunto bien definido de roles y privilegios.
- Los metadatos de roles y permisos están protegidos contra su manipulación.

2.4.6 Implementación de controles de identidad y autenticación

La autenticación se corresponde con el proceso de verificar que un individuo o una entidad es quien dice ser. La autenticación se realiza, comúnmente, enviando un nombre de usuario o ID y uno o más elementos de información privada, que solo un usuario determinado debe conocer.

La gestión de sesiones es un proceso mediante el cual un servidor mantiene el estado de una entidad que interactúa con ella. Esto resulta necesario para que un servidor recuerde cómo reaccionar a las solicitudes posteriores durante una transacción. Las sesiones se mantienen en el servidor mediante un identificador de sesión, que se puede pasar de un lado a otro entre el cliente y el servidor al transmitir y recibir solicitudes. Las sesiones deben ser únicas por usuario y, computacionalmente, imposibles de predecir.

Es importante que la aplicación cubra los siguientes requisitos de alto nivel:

- Capacidad de comprobar la identidad de los usuarios que intentan entrar en la aplicación.
- Posibilidad de garantizar que solo los usuarios autorizados sean capaces de autenticarse y que las credenciales del usuario se transporten por la red de forma segura.

En cuanto a los **protocolos de autenticación**, mientras que la autenticación se realice a través de una combinación usuario/contraseña y el uso de la autenticación de factores múltiples, esta opción se considera de las más seguras hoy día. Entre los principales protocolos de autenticación, los más extendidos actualmente son:

- **OAuth**. Open Authorization (OAuth) es un protocolo que permite a una aplicación autentificar a un usuario contra un servidor, sin requerir contraseñas o algún servidor externo que actúe como proveedor de identidad. Utiliza un token generado por el servidor, y ofrece un flujo de autorización para que un cliente, tal como una aplicación móvil, pueda llamar al servidor sobre el cual el usuario está utilizando el servicio. OAuth 2.0 se basa en HTTPS para la seguridad y, actualmente, es usado e implementado por las API de empresas como Facebook, Google, Twitter y Microsoft.
- **OpenId**. OpenId es un protocolo basado en HTTP que utiliza proveedores de identidad para validar que un usuario es quien dice ser. Constituye un protocolo que permite a un proveedor de servicios de identidad un camino para el inicio de sesión único (SSO, por las siglas en inglés de *single sign-on*). Esto permite a los usuarios reutilizar la misma identidad dada a un proveedor de identidad OpenId de confianza y ser el mismo usuario en múltiples sitios web, sin necesidad de proveer de la contraseña a ningún sitio de servicio web externo.

En algunos casos, un nombre de usuario y una contraseña no proporcionan suficiente seguridad para una aplicación móvil. Cuando se trata de datos o transacciones confidenciales, resulta importante implementar la autenticación de dos factores. Las opciones para la autenticación basada en dos factores incluyen:

- Contraseña adicional
- Código adicional proporcionado por SMS o correo electrónico
- Contraseña más un valor adicional conocido solo por el usuario
- Pregunta/respuesta de seguridad, establecida de antemano durante el registro del usuario

2.4.7 Autenticación por múltiples factores

La autenticación por múltiples factores es el uso de más de un factor de autenticación para iniciar sesión o procesar una transacción, mediante:

- Algo que se conoce (detalles de la cuenta o contraseñas).
- Algo que se tiene (tokens o teléfonos móviles).
- Algo que se es (factores biométricos).

Los esquemas de autenticación como las contraseñas de un solo uso (OTP, por las siglas en inglés de *one time passwords*) implementadas utilizando un token físico (hardware) también pueden conformar un factor clave en la lucha contra ataques, tales como los ataques CSRF del lado del cliente.

2.4.8 Manejo de errores y excepciones

El objetivo principal del manejo y registro de errores reside en proporcionar una reacción útil por parte del usuario, los administradores y los equipos de respuesta a incidentes. Debemos asegurarnos de que una aplicación verificada cumple los siguientes requisitos de alto nivel:

- No recopilar o registrar información confidencial si no se requiere específicamente.
- Asegurarnos de que toda la información registrada sea manejada de forma segura y protegida, según las leyes de protección y privacidad de los datos.
- Si los registros contienen datos privados o confidenciales, cuya definición varía de un país a otro, los registros se convierten en la información más sensible que posee la aplicación y, por lo tanto, resultan muy atractivos para los atacantes.
- Deshabilitar errores detallados. Los errores detallados pueden ser útiles en un entorno de desarrollo, pero, en un entorno de producción, pueden filtrar información sensible de la aplicación.

El control de excepciones permite gestionar los diferentes tipos de errores que puedan producirse, tanto en el sistema como en la aplicación. Una incorrecta gestión de las excepciones, involuntariamente, puede llegar a mostrar información propia de la aplicación, la cual no debería ser accesible. Un ejemplo de una incorrecta gestión de las excepciones se produce cuando no se consigue validar correctamente algún dato de entrada enviado por el usuario y la aplicación no sabe tratar dicha excepción. Esto provoca un error, el cual muestra demasiada información, como trazas de la pila, sentencias SQL erróneas u otra información de depuración.

Se recomienda manejar, de forma adecuada, todos los posibles errores de la aplicación y que, cuando suceda un error, se responda con un resultado específicamente diseñado que sea de ayuda al usuario, pero que no revele detalles internos innecesarios.

El **control de excepciones** deberá tener las siguientes características:

- Se deberán capturar y controlar todas las excepciones que se puedan producir en la aplicación, para evitar posibles fallos de disponibilidad, control de acceso o confidencialidad.
- Se deberá realizar un tratamiento de todos los códigos de retorno, para comprobar la causa de la excepción.
- Como norma general, las operaciones dentro de una aplicación que deberán ser capturadas son las siguientes: operaciones de entrada/salida, acceso a datos, operaciones con hilos y llamadas a terceros productos.

2.5 Ataques en aplicaciones web

Los problemas de seguridad han crecido rápidamente en los últimos años. En las aplicaciones desarrolladas en C/C++, el principal vector de ataque se corresponde con el de desbordamiento de búfer, debido a que los desarrolladores no tienen en cuenta los tamaños de los búferes de datos. Este problema sigue siendo bastante común hoy día en navegadores que utilizan C/C++ como lenguaje de programación.

En la actualidad, la mayoría de las aplicaciones que se desarrollan incluyen tecnologías web, donde tenemos otros vectores de ataque, como los de inyección de SQL o cross-site scripting (XSS). Estos ataques se aprovechan de una incorrecta validación de los datos de entrada. Otros tipos de ataques, como el de CSRF de falsificación de solicitudes entre sitios, atacan tanto el contexto de entrada como el de salida de una aplicación.

El problema del contexto de salida se crea por el hecho de que la salida es interpretada y procesada de manera diferente por distintos motores de renderizado por los navegadores, además de los múltiples motores de plantillas que podemos encontrar hoy día para diferentes framework webs desarrollados en Java, Python o Ruby. Conocer el contexto de salida y saber cómo se muestran los datos proporcionados por el usuario es, actualmente, un requisito esencial para la seguridad de las aplicaciones web, de móvil o de escritorio.

2.5.1 Vectores de ataque

Los vectores de ataque son las vías específicas que podrían hacer que un ataque se realice con éxito; por ejemplo, incluir un código de consulta MySQL dentro

de una solicitud HTTP con la posibilidad de que una inyección de SQL se realice con éxito.

Una aplicación segura debe conocer estas posibilidades y tener un código que las maneje adecuadamente. Existen muchas partes específicas de las aplicaciones web que pueden ser atacadas. Entre los **vectores de ataque de entrada**, podemos destacar:

- Encabezados HTTP
- Entradas de formulario a través del método POST
- Entradas de formulario ocultas a través del método POST
- Valores de parámetros de url a través de GET
- Solicitud de datos vía AJAX con método GET/POST

Entre los **vectores de ataque de salida**, podemos destacar:

- Texto HTML proporcionado por el usuario
- Hipervínculos y enlaces proporcionados por el usuario
- Hipervínculos proporcionados por el usuario que se ejecutan a través de JavaScript

Vulnerabilidades clásicas como inyección de SQL o XSS siguen siendo hoy día vulnerabilidades top de OWASP y las principales amenazas al trabajar con nuevas tecnologías tanto en frontend como en backend. También han surgido nuevos ataques del tipo XSS que permiten obtener cookies y credenciales del usuario.

2.5.2 Cross-site scripting (XSS)

Una aplicación web es vulnerable a XSS cuando aquello que nosotros enviamos al servidor (un comentario, un cambio en un perfil, una búsqueda, etc.) se ve posteriormente mostrado en la página de respuesta. Básicamente, un ataque XSS funciona engañando al motor de plantillas HTML que esté usando la aplicación, para ejecutar código arbitrario cuando lo que debería hacer, simplemente, sería mostrar los datos.

Por ejemplo, cuando realizamos una búsqueda y se nos muestra un mensaje (“No se han encontrado resultados”), se está incluyendo dentro de la página el mismo texto que nosotros hemos introducido. Ahí es donde podemos empezar a sospechar que el sitio web se muestra vulnerable a este tipo de ataque.

Cuando se habla de ejecución remota de código, se debe considerar cómo se realiza la interacción con la página. Evidentemente, toda petición enviada al servidor va a ser procesada por este y, en función de cómo la interprete, será o no factible un ataque mediante XSS.

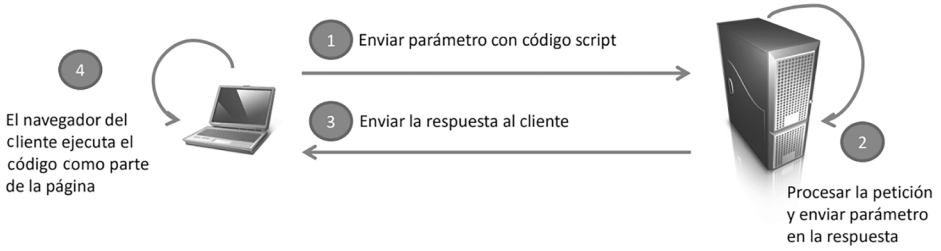


Figura 2.3 Ejemplo de ataque XSS.

Dentro de los posibles fallos de XSS, podemos distinguir dos grandes categorías:

- **No permanentes**: Esta categoría queda ilustrada con el caso que ahora se menciona. Nos encontramos con una página web que dispone de buscador, el cual, al introducir una palabra inventada o una cadena aleatoria de caracteres, muestra un mensaje del tipo: "No se han encontrado resultados para la búsqueda <texto>", donde <texto> es la cadena introducida en el campo de búsqueda. Si en la búsqueda, se introduce como <texto> el código JavaScript antes indicado, y de nuevo aparece la ventana de alerta, significa que la aplicación es vulnerable a XSS. La diferencia radica en que, en esta ocasión, los efectos de la acción no resultan permanentes.
- **Permanentes**: Su denominación se debe al hecho de que, como mostraba el ejemplo, la ventana de alerta en JavaScript queda almacenada en algún lugar, habitualmente una base de datos SQL, y se muestra a cualquier usuario que visite nuestro perfil. Evidentemente, este tipo de fallos de XSS son mucho más peligrosos que los no permanentes.

Para inyectar el código, se localiza una zona de la página que, por su funcionalidad, incorpora dentro de su propio código HTML el código JavaScript, que anteriormente se ha introducido en algún lugar de la aplicación. Si este código se ha escrito en algún campo donde queda almacenado en una base de datos de forma permanente, se mostrará cada vez que un usuario acceda a la página. Sin embargo, las vulnerabilidades más habituales son las no permanentes. En estos casos, se debe

recurrir a la ingeniería social para efectuar el ataque. Para ello, resulta necesario fijarse, en primer lugar, en la url de la página, que deberá ser algo similar a:

```
http://www.dominio_victima.com/search?query=<script>alert(“alert”);</script>
```

Vamos a comentar a continuación las posibles implicaciones de seguridad que puede presentar un fallo de este tipo. Ha de considerarse que se trata únicamente de ideas generales y que el límite lo pone la imaginación del atacante y las funcionalidades de la aplicación objetivo del ataque:

- Un ataque de XSS puede tomar el control sobre el navegador del usuario afectado y, como tal, realizar acciones en la aplicación web. Si se ha logrado que un usuario administrador ejecute nuestro código JavaScript, las posibilidades de actuación maliciosa son muy superiores. Podrá, por citar algún ejemplo, desde borrar todas las noticias de una página a generar una cuenta de administrador con los datos que nosotros especifiquemos. Si el código JavaScript se ejecuta por un usuario sin derechos de administración, se podrá realizar cualquier modificación asociada al perfil específico del usuario en el sitio web.
- Igualmente, se pueden ejecutar ataques de defacement apoyándonos en técnicas que explotan vulnerabilidades XSS.
- Por otra parte, aunque menos habitual, es posible realizar un ataque de denegación de servicios distribuidos. Para ello, se puede forzar mediante código JavaScript a que los navegadores hagan un uso intensivo de recursos muy costosos en ancho de banda o capacidad de procesamiento de un servidor de forma asíncrona.

Como se puede comprobar, las posibilidades que ofrece el XSS son realmente amplias. Las únicas limitaciones se encuentran en la imposibilidad de ejecutar código fuera del navegador, dado que la sandbox sobre la que se ejecuta no permite el acceso a ficheros del sistema y a las propias funcionalidades que ofrezca el sitio web objeto del posible ataque.

2.5.3 Cross-site request forgery (CSRF)

CSRF (falsificación de solicitudes en sitios cruzados) se basa en valores de formulario conocidos o predecibles y en una sesión de navegador iniciada. Cada